RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Department of Artificial Intelligence

V Semester B. Tech AIDS

AD23532 - Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 1: Data Preprocessing and Cleaning

Register Number: 241801157 **Date:** 06-10-2026

Dataset: Telco Customer Churn (IBM) **Tools:** Python, Pandas, NumPy, Matplotlib, Seaborn, scikit-learn

Aim

To preprocess the Telco Customer Churn dataset by handling missing values, removing inconsistencies and duplicates, encoding categorical features, standardising numerical attributes and visualising important relationships, so that the data is ready for building a churn prediction model.

Task 1: Import the required Python libraries

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder, StandardScaler
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='whitegrid')
pd.set_option('display.max_columns', None)

print("Libraries imported successfully")
print("pandas:", pd.__version__, "| numpy:", np.__version__, "| seaborn:", sns.__version__)
```

Out [1]:

```
Libraries imported successfully
pandas: 3.0.2 | numpy: 2.4.4 | seaborn: 0.13.2
```

Task 2: Load the Telco Customer Churn dataset into a Pandas DataFrame

In [2]:

```
df = pd.read_csv('Telco-Customer-Churn.csv')
print("Dataset loaded successfully")
```

Out [2]:

```
Dataset loaded successfully
```

2.1 First 10 records

In [3]:

```
df_head = df.head(10)
display(df_head)
```

Out [3]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	Yes
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	No
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	No
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	No
5	9305-CDSKC	Female	0	No	No	8	Yes	Yes	Fiber optic	No	No

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup
6	1452-KIOVK	Male	0	No	Yes	22	Yes	Yes	Fiber optic	No	Yes
7	6713-OKOMC	Female	0	No	No	10	No	No phone service	DSL	Yes	No
8	7892-POOKP	Female	0	Yes	No	28	Yes	Yes	Fiber optic	No	No
9	6388-TABGU	Male	0	No	Yes	62	Yes	No	DSL	Yes	Yes

	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	No	No	No	No	Month-to-month	Yes	Electronic check	29.85	29.85	No
1	Yes	No	No	No	One year	No	Mailed check	56.95	1889.5	No
2	No	No	No	No	Month-to-month	Yes	Mailed check	53.85	108.15	Yes
3	Yes	Yes	No	No	One year	No	Bank transfer (automatic)	42.3	1840.75	No
4	No	No	No	No	Month-to-month	Yes	Electronic check	70.7	151.65	Yes
5	Yes	No	Yes	Yes	Month-to-month	Yes	Electronic check	99.65	820.5	Yes
6	No	No	Yes	No	Month-to-month	Yes	Credit card (automatic)	89.1	1949.4	No
7	No	No	No	No	Month-to-month	No	Mailed check	29.75	301.9	No
8	Yes	Yes	Yes	Yes	Month-to-month	Yes	Electronic check	104.8	3046.05	Yes
9	No	No	No	No	One year	No	Bank transfer (automatic)	56.15	3487.95	No

2.2 Last 10 records

In [4]:

```
df_tail = df.tail(10)
display(df_tail)
```

Out [4]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup
7033	9767-FFLEM	Male	0	No	No	38	Yes	No	Fiber optic	No	No
7034	0639-TSIQW	Female	0	No	No	67	Yes	Yes	Fiber optic	Yes	Yes
7035	8456-QDAVC	Male	0	No	No	19	Yes	No	Fiber optic	No	No
7036	7750-EYXWZ	Female	0	No	No	12	No	No phone service	DSL	No	Yes
7037	2569-WGERO	Female	0	No	No	72	Yes	No	No	No internet service	No internet service
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL	Yes	No
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	No	Yes
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No phone service	DSL	Yes	No
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic	No	No
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic	Yes	No

	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	PaymentMethod
7033	No	No	No	No	Month-to-month	Yes	Credit card (automatic)
7034	Yes	No	Yes	No	Month-to-month	Yes	Credit card (automatic)
7035	No	No	Yes	No	Month-to-month	Yes	Bank transfer (automatic)
7036	Yes	Yes	Yes	Yes	One year	No	Electronic check
7037	No internet service	No internet service	No internet service	No internet service	Two year	Yes	Bank transfer (automatic)
7038	Yes	Yes	Yes	Yes	One year	Yes	Mailed check
7039	Yes	No	Yes	Yes	One year	Yes	Credit card (automatic)
7040	No	No	No	No	Month-to-month	Yes	Electronic check
7041	No	No	No	No	Month-to-month	Yes	Mailed check
7042	Yes	Yes	Yes	Yes	Two year	Yes	Bank transfer (automatic)

	MonthlyCharges	TotalCharges	Churn
7033	69.5	2625.25	No
7034	102.95	6886.25	Yes
7035	78.7	1495.1	No
7036	60.65	743.3	No
7037	21.15	1419.4	No
7038	84.8	1990.5	No
7039	103.2	7362.9	No
7040	29.6	346.45	No
7041	74.4	306.6	Yes
7042	105.65	6844.5	No

2.3 Shape of the dataset

In [5]:

```
print("Shape of the dataset:", df.shape)
print("Number of rows   :", df.shape[0])
print("Number of columns:", df.shape[1])
```

Out [5]:

```
Shape of the dataset: (7043, 21)
Number of rows   : 7043
Number of columns: 21
```

2.4 Column names

In [6]:

```
print("Column names:")
for i, col in enumerate(df.columns, start=1):
    print(f"{i:2d}. {col}")
```

Out [6]:

```
Column names:
 1. customerID
 2. gender
 3. SeniorCitizen
 4. Partner
 5. Dependents
 6. tenure
 7. PhoneService
 8. MultipleLines
 9. InternetService
10. OnlineSecurity
11. OnlineBackup
12. DeviceProtection
13. TechSupport
14. StreamingTV
15. StreamingMovies
16. Contract
17. PaperlessBilling
18. PaymentMethod
19. MonthlyCharges
20. TotalCharges
21. Churn
```

Task 3: Explore the dataset

3.1 Dataset information

In [7]:

```
import io
buf = io.StringIO()
df.info(buf=buf)
print(buf.getvalue())
```

Out [7]:

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   str
 1   gender            7043 non-null   str
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   str
 4   Dependents        7043 non-null   str
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   str
 7   MultipleLines     7043 non-null   str
 8   InternetService   7043 non-null   str
 9   OnlineSecurity    7043 non-null   str
 10  OnlineBackup      7043 non-null   str
 11  DeviceProtection  7043 non-null   str
 12  TechSupport       7043 non-null   str
 13  StreamingTV       7043 non-null   str
 14  StreamingMovies   7043 non-null   str
 15  Contract          7043 non-null   str
 16  PaperlessBilling  7043 non-null   str
 17  PaymentMethod     7043 non-null   str
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   str
 20  Churn             7043 non-null   str
dtypes: float64(1), int64(2), str(18)
memory usage: 1.1 MB
```

3.2 Summary statistics (numerical columns)

In [8]:

```
display(df.describe().round(2))
```

Out [8]:

	SeniorCitizen	tenure	MonthlyCharges
count	7043.0	7043.0	7043.0
mean	0.16	32.37	64.76
std	0.37	24.56	30.09
min	0.0	0.0	18.25
25%	0.0	9.0	35.5
50%	0.0	29.0	70.35
75%	0.0	55.0	89.85
max	1.0	72.0	118.75

3.2 Summary statistics (categorical columns)

In [9]:

```
display(df.describe(exclude='number').T)
```

Out [9]:

	count	unique	top	freq
customerID	7043	7043	7590-VHVEG	1
gender	7043	2	Male	3555
Partner	7043	2	No	3641
Dependents	7043	2	No	4933
PhoneService	7043	2	Yes	6361
MultipleLines	7043	3	No	3390
InternetService	7043	3	Fiber optic	3096
OnlineSecurity	7043	3	No	3498
OnlineBackup	7043	3	No	3088
DeviceProtection	7043	3	No	3095
TechSupport	7043	3	No	3473
StreamingTV	7043	3	No	2810
StreamingMovies	7043	3	No	2785
Contract	7043	3	Month-to-month	3875
PaperlessBilling	7043	2	Yes	4171
PaymentMethod	7043	4	Electronic check	2365
TotalCharges	7043	6531	20.2	11
Churn	7043	2	No	5174

3.3 Number of missing values

In [10]:

```
missing = df.isnull().sum()
print("Missing values in each column (as read from the file):")
print(missing)
print("\nTotal missing values:", missing.sum())
```

Out [10]:

```
Missing values in each column (as read from the file):
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
Total missing values: 0
```

Checking for hidden blanks

In [11]:

```
blank_rows = df[df['TotalCharges'].astype(str).str.strip() == '']
print("Rows with a blank TotalCharges value:", len(blank_rows))
display(blank_rows[['customerID', 'tenure', 'MonthlyCharges', 'TotalCharges']])
```

Out [11]:

```
Rows with a blank TotalCharges value: 11
```

	customerID	tenure	MonthlyCharges	TotalCharges
488	4472-LVYGI	0	52.55	
753	3115-CZMZD	0	20.25	
936	5709-LVOEQ	0	80.85	
1082	4367-NUYAO	0	25.75	
1340	1371-DWPAZ	0	56.05	
3331	7644-OMVMY	0	19.85	
3826	3213-VVOLG	0	25.35	
4380	2520-SGTTA	0	20.0	
5218	2923-ARZLG	0	19.7	
6670	4075-WKNIU	0	73.35	
6754	2775-SEFEE	0	61.9	

> **Observation:** isnull() reports no missing values, but TotalCharges is stored as text because 11 rows contain a blank string. All 11 rows have tenure = 0, i.e. customers who have not completed a billing cycle yet.

Fixing the data type of TotalCharges

In [12]:

```
text_cols = df.select_dtypes(exclude='number').columns
df[text_cols] = df[text_cols].apply(lambda s: s.astype(str).str.strip())
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

missing = df.isnull().sum()
print("Missing values after correcting the data type:")
print(missing[missing > 0])
print("\nTotal missing values:", missing.sum())
```

Out [12]:

```
Missing values after correcting the data type:
TotalCharges    11
dtype: int64

Total missing values: 11
```

Checking category labels for inconsistencies

In [13]:

```
for col in ['gender', 'Partner', 'Dependents', 'Contract', 'PaymentMethod', 'InternetService', 'Churn']:
    print(f"{col:16s}: {df[col].unique().tolist()}")
```

Out [13]:

```
gender          : ['Female', 'Male']
Partner         : ['Yes', 'No']
Dependents      : ['No', 'Yes']
Contract        : ['Month-to-month', 'One year', 'Two year']
PaymentMethod   : ['Electronic check', 'Mailed check', 'Bank transfer (automatic)', 'Credit card (automatic)']
InternetService : ['DSL', 'Fiber optic', 'No']
Churn           : ['No', 'Yes']
```

3.4 Number of duplicate records

In [14]:

```
print("Duplicate rows (all columns):", df.duplicated().sum())
print("Duplicate customerID values :", df['customerID'].duplicated().sum())
```

Out [14]:

```
Duplicate rows (all columns): 0
Duplicate customerID values : 0
```

Task 4: Clean the data

4(a) Remove duplicate records

In [15]:

```
rows_before = df.shape[0]
df = df.drop_duplicates()
df = df.drop_duplicates(subset='customerID', keep='first')
print("Rows before removing duplicates:", rows_before)
print("Rows after removing duplicates :", df.shape[0])
print("Duplicate records removed      :", rows_before - df.shape[0])
```

Out [15]:

```
Rows before removing duplicates: 7043
Rows after removing duplicates : 7043
Duplicate records removed      : 0
```

4(b) Replace missing values (numerical: median, categorical: mode)

In [16]:

```
num_cols = df.select_dtypes(include='number').columns
cat_cols = df.select_dtypes(exclude='number').columns

for col in num_cols:                        # numerical -> median
    n = df[col].isnull().sum()
    if n > 0:
        med = df[col].median()
        df[col] = df[col].fillna(med)
        print(f"[Numerical] {col}: {n} missing values replaced with median = {med:.2f}")

for col in cat_cols:                        # categorical -> mode
    n = df[col].isnull().sum()
    if n > 0:
        mode = df[col].mode()[0]
        df[col] = df[col].fillna(mode)
        print(f"[Categorical] {col}: {n} missing values replaced with mode = {mode}")

print("\nNumerical columns:", list(num_cols))
```

Out [16]:

```
[Numerical] TotalCharges: 11 missing values replaced with median = 1397.47

Numerical columns: ['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges']
```

4(c) Verify that no missing values remain

In [17]:

```
remaining = df.isnull().sum()
print("Total missing values remaining:", remaining.sum())
if remaining.sum() == 0:
    print("Verified: the dataset has no missing values.")
df_clean = df.copy()      # cleaned data in original units, used for plots
print("Cleaned dataset shape:", df_clean.shape)
```

Out [17]:

```
Total missing values remaining: 0
Verified: the dataset has no missing values.
Cleaned dataset shape: (7043, 21)
```

Task 5: Encode categorical columns

Label encoding for binary columns (gender, Partner, Dependents, Churn); ordinal encoding for Contract (natural order: month-to-month < one year < two year); one-hot encoding for PaymentMethod and InternetService (no natural order).

(a) Label encoding

In [18]:

```
df_encoded = df_clean.copy()
binary_cols = ['gender', 'Partner', 'Dependents', 'Churn']
le = LabelEncoder()
for col in binary_cols:
    df_encoded[col] = le.fit_transform(df_encoded[col])
    print(f"{col:11s}:", {c: i for i, c in enumerate(le.classes_)})
```

Out [18]:

```
gender     : {'Female': 0, 'Male': 1}
Partner    : {'No': 0, 'Yes': 1}
Dependents : {'No': 0, 'Yes': 1}
Churn      : {'No': 0, 'Yes': 1}
```

(b) Ordinal encoding

In [19]:

```
contract_order = [['Month-to-month', 'One year', 'Two year']]
oe = OrdinalEncoder(categories=contract_order)
df_encoded['Contract'] = oe.fit_transform(df_encoded[['Contract']]).astype(int)
print("Contract:", {c: i for i, c in enumerate(contract_order[0])})
```

Out [19]:

```
Contract: {'Month-to-month': 0, 'One year': 1, 'Two year': 2}
```

(c) One-hot encoding

In [20]:

```
df_encoded = pd.get_dummies(df_encoded, columns=['PaymentMethod', 'InternetService'], dtype=int)
onehot_cols = [c for c in df_encoded.columns if c.startswith(('PaymentMethod_', 'InternetService_'))]
print("New one-hot columns:")
for c in onehot_cols:
    print("  ", c)
print("\nShape after encoding:", df_encoded.shape)
```

Out [20]:

```
New one-hot columns:
   PaymentMethod_Bank transfer (automatic)
   PaymentMethod_Credit card (automatic)
   PaymentMethod_Electronic check
   PaymentMethod_Mailed check
   InternetService_DSL
   InternetService_Fiber optic
   InternetService_No

Shape after encoding: (7043, 26)
```

Encoded columns: first 10 records

In [21]:

```
encoded_cols = binary_cols + ['Contract'] + onehot_cols
display(df_encoded[encoded_cols].head(10))
```

Out [21]:

	gender	Partner	Dependents	Churn	Contract	PaymentMethod_Bank transfer (automatic)	PaymentMethod_Credit card (automatic)	PaymentMethod_Electronic check
0	0	1	0	0	0	0	0	1
1	1	0	0	0	1	0	0	0
2	1	0	0	1	0	0	0	0
3	1	0	0	0	1	1	0	0
4	0	0	0	1	0	0	0	1
5	0	0	0	1	0	0	0	1
6	1	0	1	0	0	0	1	0
7	0	0	0	0	0	0	0	0
8	0	1	0	1	0	0	0	1
9	1	0	1	0	1	1	0	0

	PaymentMethod_Mailed check	InternetService_DSL	InternetService_Fiber optic	InternetService_No
0	0	1	0	0
1	1	1	0	0
2	1	1	0	0
3	0	1	0	0
4	0	0	1	0
5	0	0	1	0
6	0	0	1	0
7	1	1	0	0
8	0	0	1	0
9	0	1	0	0

Task 6: Standardise the numerical features using StandardScaler

In [22]:

```
scale_cols = ['tenure', 'MonthlyCharges', 'TotalCharges']
print("Before scaling:")
display(df_encoded[scale_cols].describe().round(2))

scaler = StandardScaler()
df_encoded[scale_cols] = scaler.fit_transform(df_encoded[scale_cols])

print("After scaling (mean = 0, standard deviation = 1):")
display(df_encoded[scale_cols].describe().round(2))
display(df_encoded[scale_cols].head(10))
```

Out [22]:

```
Before scaling:
After scaling (mean = 0, standard deviation = 1):
```

	tenure	MonthlyCharges	TotalCharges
count	7043.0	7043.0	7043.0
mean	32.37	64.76	2281.92
std	24.56	30.09	2265.27
min	0.0	18.25	18.8
25%	9.0	35.5	402.22
50%	29.0	70.35	1397.48
75%	55.0	89.85	3786.6
max	72.0	118.75	8684.8

	tenure	MonthlyCharges	TotalCharges
count	7043.0	7043.0	7043.0
mean	-0.0	-0.0	-0.0
std	1.0	1.0	1.0
min	-1.32	-1.55	-1.0
25%	-0.95	-0.97	-0.83
50%	-0.14	0.19	-0.39
75%	0.92	0.83	0.66
max	1.61	1.79	2.83

	tenure	MonthlyCharges	TotalCharges
0.00	-1.2774445836787656	-1.1603229160349193	-0.9942419353101233
1.00	0.06632741908223598	-0.2596289419448806	-0.17324412574959375
2.00	-1.2367242199587352	-0.3626603559551803	-0.9596740712020441
3.00	0.5142514200025698	-0.7465354629935549	-0.19476626336477718
4.00	-1.2367242199587352	0.19736523310080326	-0.9404697022531111
5.00	-0.9924020376385531	1.1595456961969886	-0.6451859741727949
6.00	-0.4223169455581282	0.8089065291619363	-0.1467994889670401 8
7.00	-0.9109613101984924	-1.1636465100352518	-0.8741373704237511
8.00	-0.1779947632379461	1.330710787214099	0.3373492744317471
9.00	1.2064976032430856	-0.2862176939475387 6	0.5324391741681479

Task 7: Distribution of MonthlyCharges and Tenure

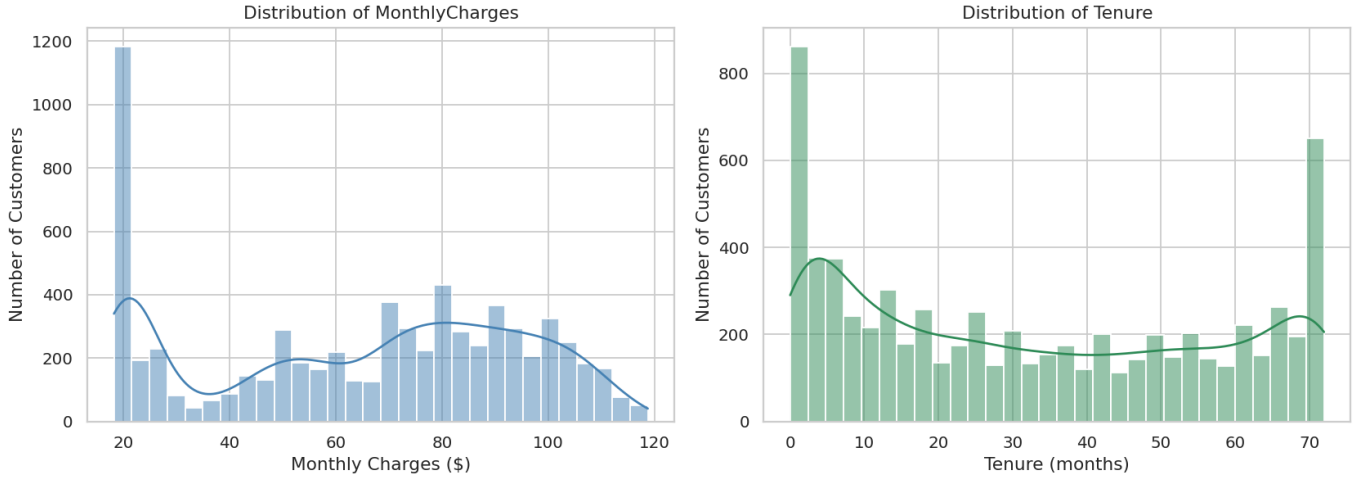
In [23]:

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
sns.histplot(df_clean['MonthlyCharges'], bins=30, kde=True, color='steelblue', ax=axes[0])
axes[0].set_title('Distribution of MonthlyCharges')
axes[0].set_xlabel('Monthly Charges ($)'); axes[0].set_ylabel('Number of Customers')
sns.histplot(df_clean['tenure'], bins=30, kde=True, color='seagreen', ax=axes[1])
axes[1].set_title('Distribution of Tenure')
axes[1].set_xlabel('Tenure (months)'); axes[1].set_ylabel('Number of Customers')
plt.tight_layout()
plt.show()

for c in ['MonthlyCharges', 'tenure']:
    print(f"{c:15s} mean: {df_clean[c].mean():.2f}, median: {df_clean[c].median():.2f}, skewness: {df_clean[c].skew():.2f}")
```

Out [23]:

```
MonthlyCharges  mean: 64.76, median: 70.35, skewness: -0.22
tenure          mean: 32.37, median: 29.00, skewness: 0.24
```



Observation: MonthlyCharges is bimodal: a large block of customers pays about $20 (phone-only or basic plans) while the rest cluster between $70 and $100, which pulls the mean (64.76) below the median (70.35). Tenure is U-shaped, with many brand-new customers (0-5 months) and a second peak of long-term customers at about 70 months.

Task 8: Count plot of Contract vs Churn

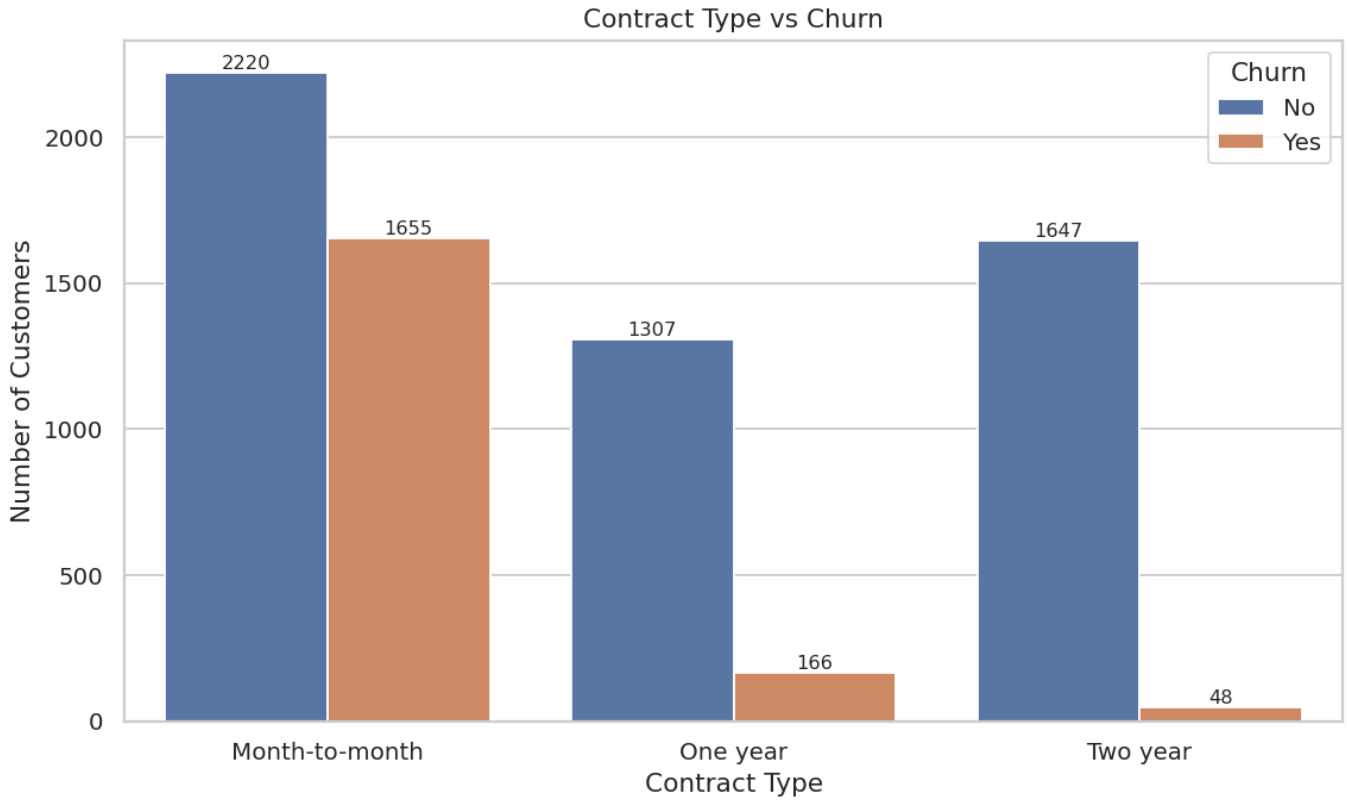
In [24]:

```
churn_palette = {'No': '#4C72B0', 'Yes': '#DD8452'}
plt.figure(figsize=(9, 5.5))
ax = sns.countplot(data=df_clean, x='Contract', hue='Churn', palette=churn_palette,
                   order=['Month-to-month', 'One year', 'Two year'], hue_order=['No', 'Yes'])
for container in ax.containers:
    ax.bar_label(container, fontsize=9)
plt.title('Contract Type vs Churn')
plt.xlabel('Contract Type'); plt.ylabel('Number of Customers')
plt.tight_layout()
plt.show()

churn_rate = pd.crosstab(df_clean['Contract'], df_clean['Churn'], normalize='index') * 100
print("Churn rate (%) by contract type:")
print(churn_rate.round(2))
```

Out [24]:

```
Churn rate (%) by contract type:
Churn              No    Yes
Contract
Month-to-month  57.29  42.71
One year        88.73  11.27
Two year        97.17   2.83
```



Observation: Month-to-month customers churn at 42.71%, compared with 11.27% for one-year and 2.83% for two-year contracts. Longer contracts are strongly associated with retention.

Task 9: Box plot of MonthlyCharges grouped by Churn

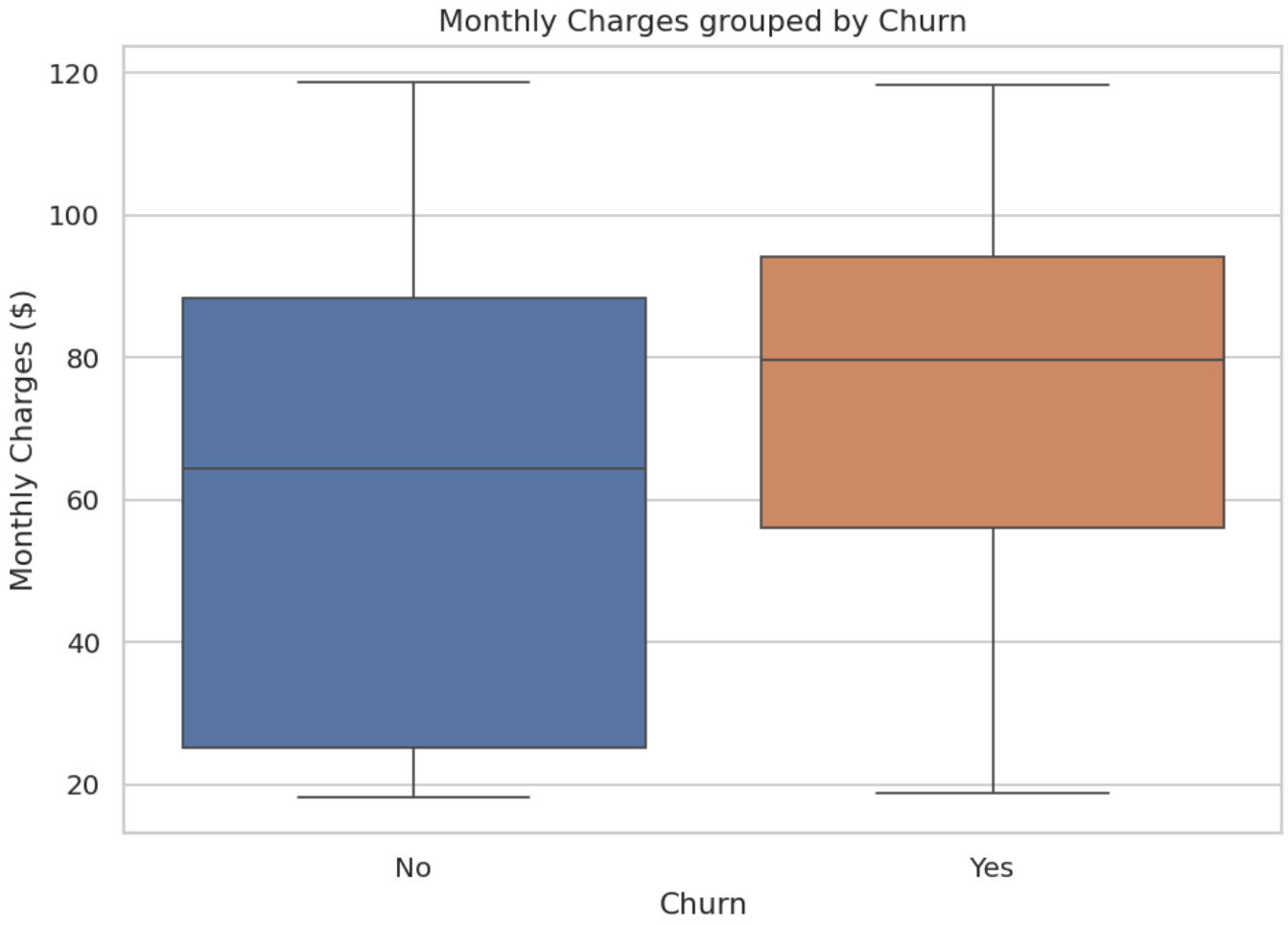
In [25]:

```
plt.figure(figsize=(7.5, 5.5))
sns.boxplot(data=df_clean, x='Churn', y='MonthlyCharges', hue='Churn',
            palette=churn_palette, order=['No', 'Yes'], legend=False)
plt.title('Monthly Charges grouped by Churn')
plt.xlabel('Churn'); plt.ylabel('Monthly Charges ($)')
plt.tight_layout()
plt.show()

print(df_clean.groupby('Churn')['MonthlyCharges'].describe().round(2))
```

Out [25]:

```
       count   mean    std    min    25%    50%   75%     max
Churn
No     5174.0  61.27  31.09  18.25  25.10  64.43  88.4  118.75
Yes    1869.0  74.44  24.67  18.85  56.15  79.65  94.2  118.35
```



> **Observation:** Customers who churned have a higher median monthly charge ($79.65) than those who stayed ($64.43), and the churned group has far fewer low-priced plans.

Task 10: Correlation heatmap of all numerical features

After encoding and scaling every feature used here is numerical, so the heatmap covers all numerical columns of the preprocessed dataset.

In [26]:

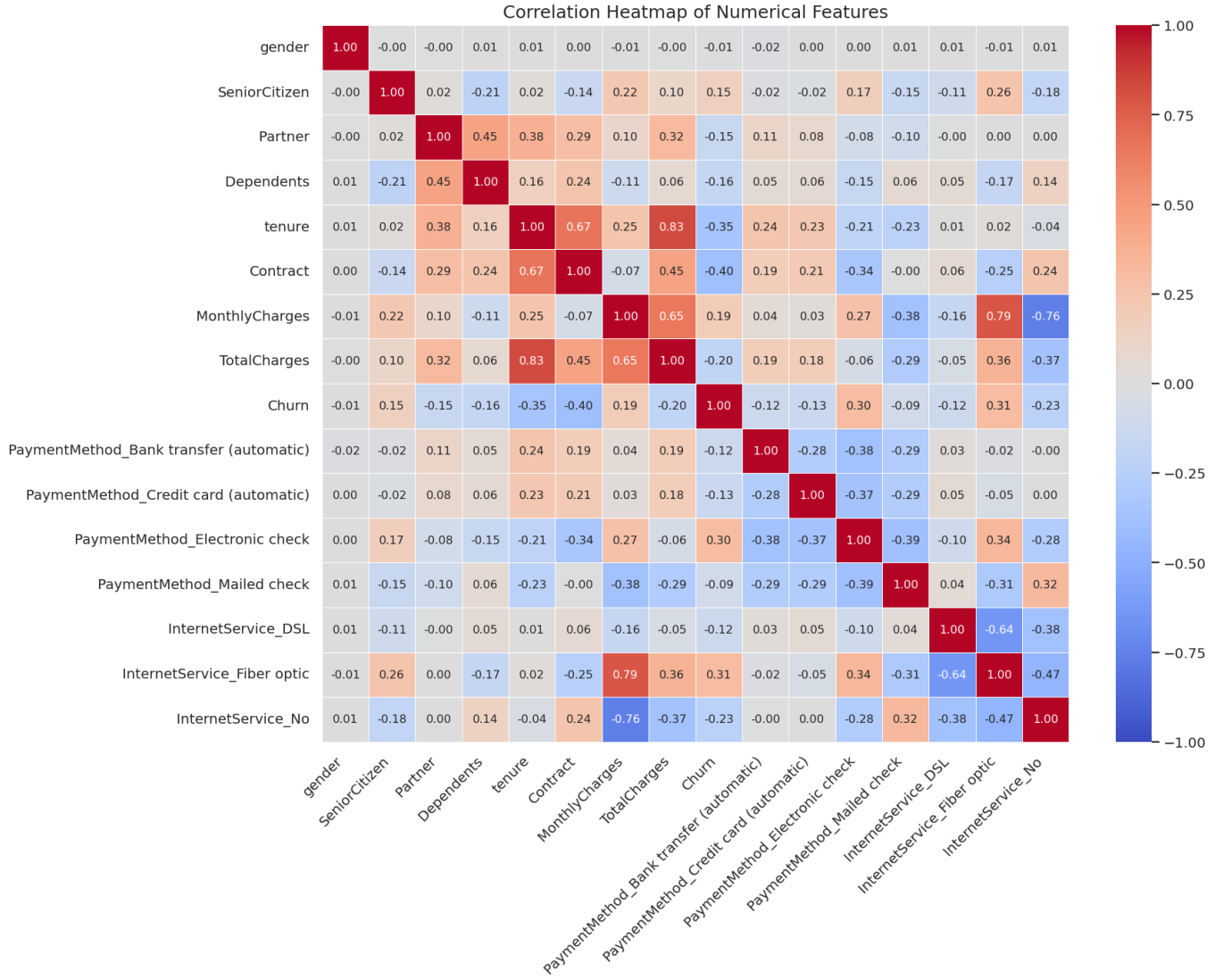
```
numeric_df = df_encoded.select_dtypes(include='number')
corr = numeric_df.corr()

plt.figure(figsize=(14, 11))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1, center=0,
            linewidths=0.5, annot_kws={'size': 9})
plt.title('Correlation Heatmap of Numerical Features', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

print(corr['Churn'].drop('Churn').sort_values(ascending=False).round(3))
```

Out [26]:

```
InternetService_Fiber optic               0.308
PaymentMethod_Electronic check            0.302
MonthlyCharges                            0.193
SeniorCitizen                             0.151
gender                                   -0.009
PaymentMethod_Mailed check               -0.092
PaymentMethod_Bank transfer (automatic)  -0.118
InternetService_DSL                      -0.124
PaymentMethod_Credit card (automatic)    -0.134
Partner                                  -0.150
Dependents                               -0.164
TotalCharges                             -0.199
InternetService_No                       -0.228
tenure                                   -0.352
Contract                                 -0.397
Name: Churn, dtype: float64
```



Observation: Churn is most negatively correlated with Contract (-0.40) and tenure (-0.35), and most positively with fiber optic internet (0.31) and electronic check payment (0.30). TotalCharges is strongly correlated with tenure (0.83) because it accumulates over time.

Result

Thus the Telco Customer Churn dataset (7,043 records, 21 attributes) was preprocessed using Python. The data was explored, the hidden blank values in TotalCharges were converted to missing values and replaced with the median, duplicates were checked and removed, the categorical columns were encoded using label, ordinal and one-hot encoding, the numerical features were standardised using StandardScaler, and the key relationships were visualised. The cleaned dataset is ready for building a churn prediction model.